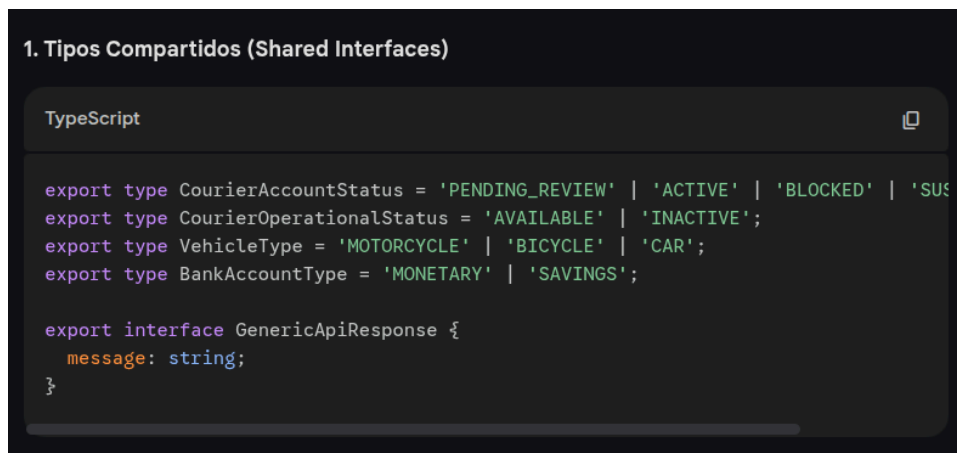


ADMINISTRACIÓN Y CONTABILIDAD

Equipo responsable: Servicio de Administración y Contabilidad



```
export type CourierAccountStatus = 'PENDING_REVIEW' | 'ACTIVE' | 'BLOCKED' | 'SUSPENDED_DEBT';
```

Endpoint: **POST** </api/couriers/register>

- **Descripción:** Recibe y procesa la solicitud de afiliación de un nuevo repartidor.
- **Por qué lo necesitamos:** El frontend recopila un formulario multipaso (Wizard) con los siguientes datos personales:

- Nombre
- Apellidos
- Fecha de nacimiento
- Nacionalidad
- Departamento de residencia
- Dirección de domicilio
- Teléfono
- Correo electrónico (usuario)
- CUI
- Fotografía del DPI
- NIT
- Fotografía
- Tipo de vehículo
- Placa
- Crear una cuenta bancaria
 - Seleccionar el tipo de cuenta
 - Seleccionar el banco

Al crear la cuenta de banco, se crea automáticamente una Tarjeta de crédito o débito asociada al colaborador

- Contraseña

Necesitamos enviar este payload masivo para que un administrador evalúe el perfil y determine si aprueba o rechaza la solicitud .

- **Ejemplo:**

TypeScript Interfaces:

TypeScript

```
export interface CourierRegistrationRequest {  
  firstName: string;  
  lastName: string;  
  birthDate: string;  
  nationality: string;  
  department: string;  
  address: string;  
  phone: string;  
  email: string;  
  cui: string;  
  dpiPhotoBase64: string;  
  nit: string;  
  profilePhotoBase64: string;  
  vehicleType: VehicleType;  
  licensePlate: string;  
  bankAccountType: BankAccountType;  
  bankId: string;  
  passwordRaw: string;  
}  
// Response utiliza GenericApiResponse
```

Mocks JSON:

- Request:

JSON

```
{  
  "firstName": "Carlos",  
  "lastName": "Méndez",  
  "birthDate": "1995-08-14",  
  "nationality": "Guatemalteca",  
  "department": "Quetzaltenango",  
  "address": "Zona 3, Quetzaltenango",  
  "phone": "5555-1234",  
  "email": "carlos.m@ejemplo.com",  
  "cui": "2541876540901",  
  "dpiPhotoBase64": "data:image/jpeg;base64,/9j/4AAQSkZJ...",  
  "nit": "9876543-2",  
  "profilePhotoBase64": "data:image/jpeg;base64,/9j/4AAQSkZJ...",  
  "vehicleType": "MOTORCYCLE",  
  "licensePlate": "M-123ABC",  
  "bankAccountType": "SAVINGS",  
  "bankId": "BANCO_INDUSTRIAL_01",  
  "passwordRaw": "MiPasswordSegura123!"  
}
```

- Response (201 Created):

JSON

```
{  
  "message": "Solicitud de afiliación enviada exitosamente. Tu cuenta está pe  
}
```

Endpoint: `GET /api/couriers/me/account-status`

- **Descripción:** Consulta el estado administrativo de la cuenta del repartidor (ej. PENDING_REVIEW, ACTIVE, BLOCKED).
- **Por qué lo necesitamos:** El frontend debe saber si renderizar la vista estática de "Solicitud en Revisión", permitir el ingreso al Layout Principal, o restringir el acceso y mostrar alertas si el administrador ha bloqueado la cuenta.
- **Ejemplo:**

TypeScript Interfaces:

TypeScript



```
export interface AccountStatusResponse {  
  status: CourierAccountStatus;  
}
```

Mocks JSON:

- **Request:** *(Sin body)*
- **Response (200 OK):**

JSON



```
{  
  "status": "PENDING_REVIEW"  
}
```

Endpoint: `PATCH /api/couriers/me/availability`

- **Descripción:** Actualiza el estado operativo actual del repartidor dentro de la plataforma (alternando entre "Disponible" e "Inactivo").
- **Por qué lo necesitamos:** El Dashboard Principal de la interfaz exige un "Toggle de Estado" para el usuario. Requerimos este endpoint para notificar al sistema cada vez que el repartidor cambie este interruptor, ya que de esto depende que el backend lo contemple para inyectarle el feed de pedidos cercanos o lo excluya de la distribución.
- **Ejemplo:**

TypeScript Interfaces:

TypeScript



```
export interface UpdateAvailabilityRequest {  
  operationalStatus: CourierOperationalStatus;  
}  
// Response utiliza GenericApiResponse
```

Mocks JSON:

- **Request:**

JSON



```
{  
  "operationalStatus": "AVAILABLE"  
}
```

- **Response (200 OK):**

JSON



```
{  
  "message": "Estado operativo actualizado a Disponible."  
}
```

Endpoint: [GET /api/couriers/me](#)

- **Descripción:** Recupera la información detallada del perfil del repartidor autenticado, incluyendo sus datos personales, documento de identificación y detalles del vehículo.
- **Por qué lo necesitamos:** En el Módulo de Perfil, la interfaz exige renderizar "Formularios de edición para actualizar datos personales y vehículos". Para que el repartidor pueda editar su información, el Frontend necesita este endpoint de lectura para pre-llenar los campos del formulario con los datos actuales que el usuario tiene registrados en la base de datos, previo a que intente modificarlos a través del endpoint [PUT /api/couriers/me](#).
- **Ejemplo:**

TypeScript Interfaces:

TypeScript



```
export interface CourierProfileResponse {  
  firstName: string;  
  lastName: string;  
  address: string;  
  phone: string;  
  email: string;  
  vehicleType: VehicleType;  
  licensePlate: string;  
}
```

Mocks JSON:

- **Request:** (Sin body)
- **Response (200 OK):**

JSON



```
{  
  "firstName": "Carlos",  
  "lastName": "Méndez",  
  "address": "Zona 3, Quetzaltenango",  
  "phone": "5555-1234",  
  "email": "carlos.m@ejemplo.com",  
  "vehicleType": "MOTORCYCLE",  
  "licensePlate": "M-123ABC"  
}
```

Endpoint: **PUT** /api/couriers/me

- **Descripción:** Actualiza la información personal y del vehículo del repartidor.
- **Por qué lo necesitamos:** En el Módulo de Perfil, el usuario debe poder gestionar su cuenta, lo cual incluye formularios de edición para actualizar sus datos personales y vehículos.
- **Ejemplo:**

TypeScript Interfaces:

TypeScript

```
export interface UpdateProfileRequest {  
  address: string;  
  phone: string;  
  vehicleType: VehicleType;  
  licensePlate: string;  
}  
// Response utiliza GenericApiResponse
```

Mocks JSON:

- **Request:**

JSON

```
{  
  "address": "Zona 1, Quetzaltenango",  
  "phone": "5555-9999",  
  "vehicleType": "MOTORCYCLE",  
  "licensePlate": "M-999XYZ"  
}
```

- **Response (200 OK):**

JSON

```
{  
  "message": "Perfil actualizado correctamente."  
}
```

Endpoint: `POST /api/couriers/me/unlock-request`

- **Descripción:** Envía una solicitud formal de revisión para desbloquear la cuenta.
- **Por qué lo necesitamos:** La aplicación cuenta con una "Vista de Recuperación / Desbloqueo de Cuenta" que incluye un formulario específico para solicitar el desbloqueo al administrador en caso de suspensiones.
- **Ejemplo:**

TypeScript Interfaces:

TypeScript



```
export interface UnlockAccountRequest {  
  reason: string;  
}  
// Response utiliza GenericApiResponse
```

Mocks JSON:

- **Request:**

JSON



```
{  
  "reason": "Ya he saldado mi deuda pendiente mediante transferencia bancaria"  
}
```

- **Response (200 OK):**

JSON



```
{  
  "message": "Tu solicitud de desbloqueo ha sido enviada al equipo de adminis"  
}
```